GUIDE 21 · ACCESS

Verify who can reach what

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=21-verify-who-can-reach-what. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a reliable answer to "which keys can call this query?" and "what can this key call?" - from the Console, from the API, and from first principles.

Access in QueryAPIGate comes from three grants that **add up**, never narrow each other:

GRANT ON A KEY OR ROLE	REACHES
`connections: ["shop"]` (or `"*"`)	every saved query on those connections, plus ad-hoc SQL and the schema browser
`collections: ["reporting"]`	every saved query currently filed in that collection - read-only
`queries: ["orders_by_status"]` (or `"*"`)	those saved queries by name - never ad-hoc SQL

A query is reachable if **any** of the three reaches it. Then other fields can still refuse a particular call: `active: false`, a passed `expires_at`, `allowed_ips`, `allowed_tables`, `rate_limit`, and the write rules (`allow_writes`, `allowed_write_ops`, `QUERYAPIGATE_ALLOW_WRITES`). The admin key reaches everything.

The one surprise to remember: **a key created without `connections` gets every connection** (`"*"`), not none. See Set up your first scoped API key.

In the Console: the Access map

Access → Access map puts every saved query (rows) against every API key and role (columns). Each cell says how that key reaches that query:

- **Q** - by name (`queries`)
- **C** - through its collection (`collections`)
- **W** - through a whole connection (`connections`)

An empty cell means no reach. A cell can show several letters - a key may reach the same query more than one way, which matters when you remove one grant and expect access to go away.

Filters narrow it down: by connection, by database type, and **Any reach / Reachable by a key / Reachable by no key**. "Reachable by no key" finds queries only the admin key can run - often fine, sometimes a forgotten grant. Pick a connection and a table to see which queries already expose that table.

The example APIs are the quickest way to see this: five keys, four collections, one key (`example-executive`) reaching two collections at once.

One query: who can reach it?

API Repository → (the query) → API Keys and **Roles** tabs list each key and role that reaches it, and how. Moving a query to another collection (**Move…**) previews exactly which keys gain or lose it before anything changes - see Group queries into a collection.

Over the API, `GET /api/v1/collections` lists each collection's queries and the keys and roles granted that collection:

shell

```
curl http://127.0.0.1:5000/api/v1/collections -H 'X-API-Key: demo-key'
```

json

```
{"items": [{"name": "examples-reporting",
            "queries": ["example_monthly_revenue", "example_revenue_by_category", "example_top_films"],
            "keys": ["example-executive", "example-reporting"], "roles": ["example-executive", "example-
reporting"]},
           ...],
 "uncollected": ["orders_by_status", "salary_list"]}
```

That's the **C** column only - keys reaching the same queries through `connections` or by name aren't listed here. For the whole picture, use the Access map, or reason it out from `GET /api/v1/api-keys` with the table above.

One key: what can it reach?

In the Console, **Access → API keys → (the key) → Details** is the key-centric view: the saved queries (APIs) it can reach and how each is granted, the connections and tables behind them, its limits and masking, and its full run history - clicking a query there opens its SQL. Each key on the list also carries a 30-day activity sparkline (and so does each API, connection, cached query and MCP tool), so "granted but never used" shows at a glance.

Ask as the key itself over the API. `GET /catalog` lists exactly the saved queries this caller can run through `/q/<name>`, and the terms it runs them on - verified with the `example-executive` key:

shell

```
curl http://127.0.0.1:5000/catalog -H 'X-API-Key: <example-executive secret>'
```

json

```
{"queries": [
   {"name": "example_kpi_active_rentals", "collection": "examples-dashboard", "cache_ttl": 30,
"can_write": false, ...},
   {"name": "example_monthly_revenue", "collection": "examples-reporting", "cache_ttl": null,
"can_write": false, ...},
   ...],
 "caller": {"name": "example-executive", "admin": false, "allow_writes": false, "allowed_write_ops":
null,
            "allowed_tables": null, "rate_limit": "300/hour", "server_rate_limit": null}}
```

Seven queries - four from one collection, three from the other - and the key's own limits. `/openapi.json` with the same key lists the same queries. This is the safest check before handing a key out: it's computed by the same code that authorizes the calls, not a separate report that could drift.

`/catalog` covers saved queries. Whether the key can also run ad-hoc SQL or browse a schema depends only on its `connections` grant.

A checklist before handing out a key

1. `connections` is set explicitly - `[]` unless it really needs ad-hoc SQL.
2. `/catalog` with the new key lists exactly what you meant, and nothing else.
3. Writes: `allow_writes` is `false`, or `allowed_write_ops` is as narrow as possible.
4. External callers: a rate limit, an expiry, and `allowed_ips` if their addresses are stable.
5. The audit log shows the key's creation as you intended.

Next steps

- Give an external partner access to exactly one query.
- Group queries into a collection.
- Read the audit log - how access got this way.